

Guide de Référence du Code — Par Rôle

Ce document fournit les extraits de code clés que chaque développeur doit comprendre et maîtriser.

1. Code du Data Engineer (Dev 2)

backend/database.py — Modèle de Données

```
# Connexion SQLite – le fichier reviews.db est créé automatiquement DATABASE_URL =
"sqlite:///./reviews.db" engine = create_engine(DATABASE_URL, connect_args={"check_same_thread":
False}) # Modèle Product : chaque avis peut être associé à un produit/service class Product(Base):
__tablename__ = "products" id = Column(Integer, primary_key=True, index=True) name =
Column(String(150), nullable=False, unique=True) description = Column(Text, default="") created_at =
Column(DateTime, default=datetime.utcnow) reviews = relationship("Review", back_populates="product") #
Relation 1-N # Modèle Review : cœur du système – stocke l'avis + les résultats de l'analyse IA class
Review(Base): __tablename__ = "reviews" id = Column(Integer, primary_key=True, index=True) source =
Column(String(50), index=True) # WhatsApp, Email, Google, Facebook, Twitter text = Column(Text,
nullable=False) # Texte brut de l'avis sentiment = Column(String(20), index=True) # Résultat IA :
Positif, Négatif, Neutre score = Column(Integer) # Intensité : 1 (très négatif) → 5 (très positif)
confidence = Column(Float, default=0.0) # Confiance du modèle : 0.0 → 1.0 keywords = Column(Text,
default="") # Mots-clés extraits par spaCy (CSV) themes = Column(Text, default="") # Thèmes détectés
(CSV) product_id = Column(Integer, ForeignKey("products.id"), nullable=True) # FK optionnel date =
Column(DateTime, default=datetime.utcnow, index=True) product = relationship("Product",
back_populates="reviews")
```

Points importants pour le Dev 2 : - `check_same_thread=False` est nécessaire car SQLite n'autorise pas les accès multi-thread par défaut, mais FastAPI est asynchrone. - `index=True` sur `source`, `sentiment`, `date` accélère les requêtes de filtrage et de tri. - Les champs `keywords` et `themes` stockent des listes sous forme de chaîne CSV ("mot1, mot2, mot3") pour simplifier le stockage SQLite.

Parsing CSV — Extrait de `main.py`

```
@app.post("/import-csv/") async def import_csv( file: UploadFile = File(...), # Fichier uploadé source:
str = Form("CSV"), # Source par défaut product_id: Optional[int] = Form(None), db: Session =
Depends(get_db), ): content = await file.read() decoded = content.decode("utf-8-sig") # Gestion du BOM
Windows reader = csv.DictReader(io.StringIO(decoded)) # Lecture comme dictionnaire count = 0 for row in
reader: text = row.get("text", "").strip() # Colonne obligatoire if not text: continue # Ignorer les
lignes vides row_source = row.get("source", source) # Colonne optionnelle a = analyze_sentiment(text) #
APPEL AU MOTEUR IA db.add(Review( source=row_source, text=text, sentiment=a["sentiment"],
score=a["score"], confidence=a["confidence"], keywords=a["keywords"], themes=a["themes"],
product_id=pid, )) count += 1 db.commit()
```

2. Code de l'IA Engineer #1 (Dev 3)

Pipeline Transformers — `nlp_engine.py`

```
from transformers import pipeline # Chargement du modèle au démarrage (une seule fois)
sentiment_analyzer = pipeline( "sentiment-analysis",
model="nlpTown/bert-base-multilingual-uncased-sentiment", ) def analyze_sentiment(text: str) -> dict:
"""Fonction principale appelée pour chaque avis.""" result = { "sentiment": "Neutre", "score": 3,
"confidence": 0.0, "keywords": "", # Rempli par Dev 4 "themes": "", # Rempli par Dev 4 } if not text or
```

```

not text.strip(): return result # Le modèle analyse et retourne un label + confiance res =
sentiment_analyzer(text[:512])[0] # Exemple : {"label": "4 stars", "score": 0.876} score_val =
int(res["label"].split()[0]) # "4 stars" → 4 confidence = round(res["score"], 3) # 0.876
result["score"] = score_val result["confidence"] = confidence # Conversion en polarité lisible if
score_val >= 4: result["sentiment"] = "Positif" elif score_val <= 2: result["sentiment"] = "Négatif"
else: result["sentiment"] = "Neutre" # Les mots-clés et thèmes sont ajoutés par les fonctions du Dev 4
kws = extract_keywords(text) result["keywords"] = ", ".join(kws) themes = detect_themes(text)
result["themes"] = ", ".join(themes) return result

```

Ce que le Dev 3 doit comprendre :

1. **Pourquoi `text[:512]` ?** — BERT a une limite de 512 tokens. Un token ≈ un mot en français. Au-delà, le modèle crashe.
2. **Pourquoi le modèle est chargé globalement ?** — Le chargement prend ~5 secondes. Si on le rechargeait à chaque requête, ce serait 100x plus lent.
3. **Le score de confiance** — Un score de 0.95 signifie que le modèle est très sûr. Un score de 0.35 signifie qu'il hésite entre plusieurs classes.
4. **Le choix du modèle** — `nlptown/bert-base-multilingual-uncased-sentiment` est spécifiquement entraîné sur des avis produits dans 6 langues, dont le français.

3. Code de l'IA Engineer #2 (Dev 4)

Extraction de Mots-clés — spaCy

```

import spacy nlp = spacy.load("fr_core_news_sm") def extract_keywords(text: str) -> list[str]: doc =
nlp(text) # Analyse syntaxique complète keywords = [] # Étape 1 : Groupes nominaux (Noun Chunks) # "Le
service client est lent" → chunk = "Le service client" for chunk in doc.noun_chunks: if
chunk.root.pos_ == "PRON": # Ignorer "je", "il", "on" continue # Retirer les déterminants : "Le service
client" → "service client" clean = " ".join( tok.text for tok in chunk if tok.pos_ not in ("DET",
"PUNCT", "ADP") ).strip() if len(clean) > 2: keywords.append(clean.lower()) # Étape 2 : Adjectifs
isolés ("professionnel", "lent", "rapide") for tok in doc: if tok.pos_ == "ADJ" and len(tok.text) > 3:
keywords.append(tok.text.lower()) # Dédoublement en gardant l'ordre seen = set() unique = [] for kw in
keywords: if kw not in seen: seen.add(kw) unique.append(kw) return unique[:8] # Max 8 mots-clés

```

Détection de Thèmes

```

THEME_KEYWORDS = { "Service Client": ["service", "support", "aide", "réponse", "réactivité", ...],
"Qualité Produit": ["produit", "qualité", "fiable", "défaut", "cassé", ...], "Prix & Tarifs": ["prix",
"tarif", "cher", "coût", "budget", ...], "Livraison": ["livraison", "délai", "rapide", "lent", "colis",
...], "Interface & UX": ["site", "interface", "application", "navigation", ...], "Communication":
["communication", "email", "message", "campagne", ...], "Résultats & ROI": ["résultat", "vente",
"trafic", "conversion", ...], "Créativité": ["créatif", "idée", "original", "innovant", "contenu",
...], } def detect_themes(text: str) -> list[str]: text_lower = text.lower() detected = [] for theme,
words in THEME_KEYWORDS.items(): if any(w in text_lower for w in words): detected.append(theme) return
detected

```

Agrégation pour le Rapport

```

from collections import Counter def aggregate_themes(reviews_data): """Compte combien de fois chaque
thème apparaît.""" counter = Counter() for r in reviews_data: for t in r["themes"].split(", "): if
t.strip(): counter[t.strip()] += 1 return dict(counter.most_common(10)) def
extract_strengths_weaknesses(reviews_data): """Sépare mots-clés positifs (forces) et négatifs
(faiblesses).""" strengths = Counter() weaknesses = Counter() for r in reviews_data: kws = [k.strip()
for k in r["keywords"].split(", ") if k.strip()] if r["sentiment"] == "Positif": strengths.update(kws)

```

```
elif r["sentiment"] == "Négatif": weaknesses.update(kws) return { "points_forts":  
dict(strengths.most_common(8)), "points_amelioration": dict(weaknesses.most_common(8)), }
```

4. Code du Développeur Frontend (Dev 5)

Structure CSS — Glassmorphism

```
/* Le secret du look premium : backdrop-filter */ .card { background: rgba(255, 255, 255, 0.04); /*  
Quasi transparent */ backdrop-filter: blur(12px); /* Flou de l'arrière-plan */ border: 1px solid  
rgba(255, 255, 255, 0.08); border-radius: 16px; transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1); }  
.card:hover { background: rgba(255, 255, 255, 0.07); /* Légèrement plus visible */ transform:  
translateY(-2px); /* Effet "flottant" */ }
```

Communication avec l'API — Axios

```
// App.jsx – Appel parallèle de tous les endpoints const fetchAll = useCallback(async () => { const  
params = selectedProduct ? { product_id: selectedProduct } : {}; const [s, t, rep, r, p] = await  
Promise.all([ axios.get(`${API}/stats/`, { params }), axios.get(`${API}/stats/timeline/`, { params }),  
axios.get(`${API}/stats/report/`, { params }), axios.get(`${API}/reviews/`, { params: { ...params,  
limit: 50 } }) ], axios.get(`${API}/products/`), []); setStats(s.data); setTimeline(t.data);  
setReport(rep.data); setReviews(r.data); setProducts(p.data); }, [selectedProduct]);
```

Graphique — Recharts (Donut)

```
<PieChart> <Pie data={chartData} innerRadius={50} outerRadius={75} paddingAngle={4} dataKey="value">  
{chartData.map((e, i) => ( <Cell key={i} fill={COLORS[e.name]} /> ))} </Pie> <Tooltip  
content={<CustomTooltip />} /> </PieChart>
```

Modal d'Ajout — Cycle Complet

```
// 1. L'utilisateur remplit le formulaire (source + texte + produit) // 2. Envoi à l'API const res =  
await axios.post(`${apiUrl}/reviews/`, { source, text, product_id: productId || null }); // 3. L'API  
appelle analyze_sentiment() (Dev 3 + Dev 4) // 4. La réponse contient sentiment, score, keywords,  
themes // 5. Affichage du résultat avec emoji + badges setResult(res.data);
```

5. Résumé des Dépendances Entre Développeurs

Développeur	Produit	Utilisé par	Dépend de
Dev 1 (Coordinateur)	<code>main.py</code> — API complète	Dev 5 (via HTTP)	Dev 2, Dev 3, Dev 4
Dev 2 (Data Engineer)	<code>database.py</code> — Modèles BDD	Dev 1 (imports ORM)	—
Dev 3 (IA Sentiment)	<code>analyze_sentiment()</code>	Dev 1 (appel à chaque avis)	Dev 4 (keywords, themes)

Dev 4 (IA NLP)	<code>extract_keywords()</code> , <code>detect_themes()</code> , agrégations	Dev 3, Dev 1	—
Dev 5 (Frontend)	Dashboard React complet	Utilisateur final	Dev 1 (via API HTTP)

Flux de données : Dev 2 fournit le stockage, Dev 3 + Dev 4 fournissent l'intelligence, Dev 1 orchestre le tout dans l'API, et Dev 5 consomme l'API pour afficher les résultats.